

Assignment 6: GUI-Based Multi-Client Chat Application Using TCP

Name: Pritidipan Saha

Roll Number: ADTU/O/2023-27/BTCS/158

1. Objective

The objective of this assignment is to convert a terminal-based TCP chat application into a graphical desktop application using Python's tkinter library. This project introduces GUI programming, event-driven architectures, and multithreading to separate blocking network calls from the main graphical interface, ensuring a responsive user experience.

2. GUI Design

The graphical interface is built using tkinter frames to logically separate the connection flow from the main chat experience. It begins with a Login window containing a username entry and a Connect button. Upon successful connection, the interface transitions to a split-pane layout featuring a ScrolledText widget for chat history, a dynamically updated Listbox for tracking online users, and a bottom control frame for inputting messages and triggering private chats.

3. System Architecture

The application operates on a client-server TCP architecture deployed within a Mininet emulated network (`sudo mn --topo single,5`). The centralized server manages message routing, user state, and persistent logging, while multiple GUI clients connect concurrently. Multithreading is heavily utilized on both ends: the server spawns a dedicated thread per connected client, and each client spawns a background daemon thread to listen for incoming network traffic without freezing the main UI event loop.

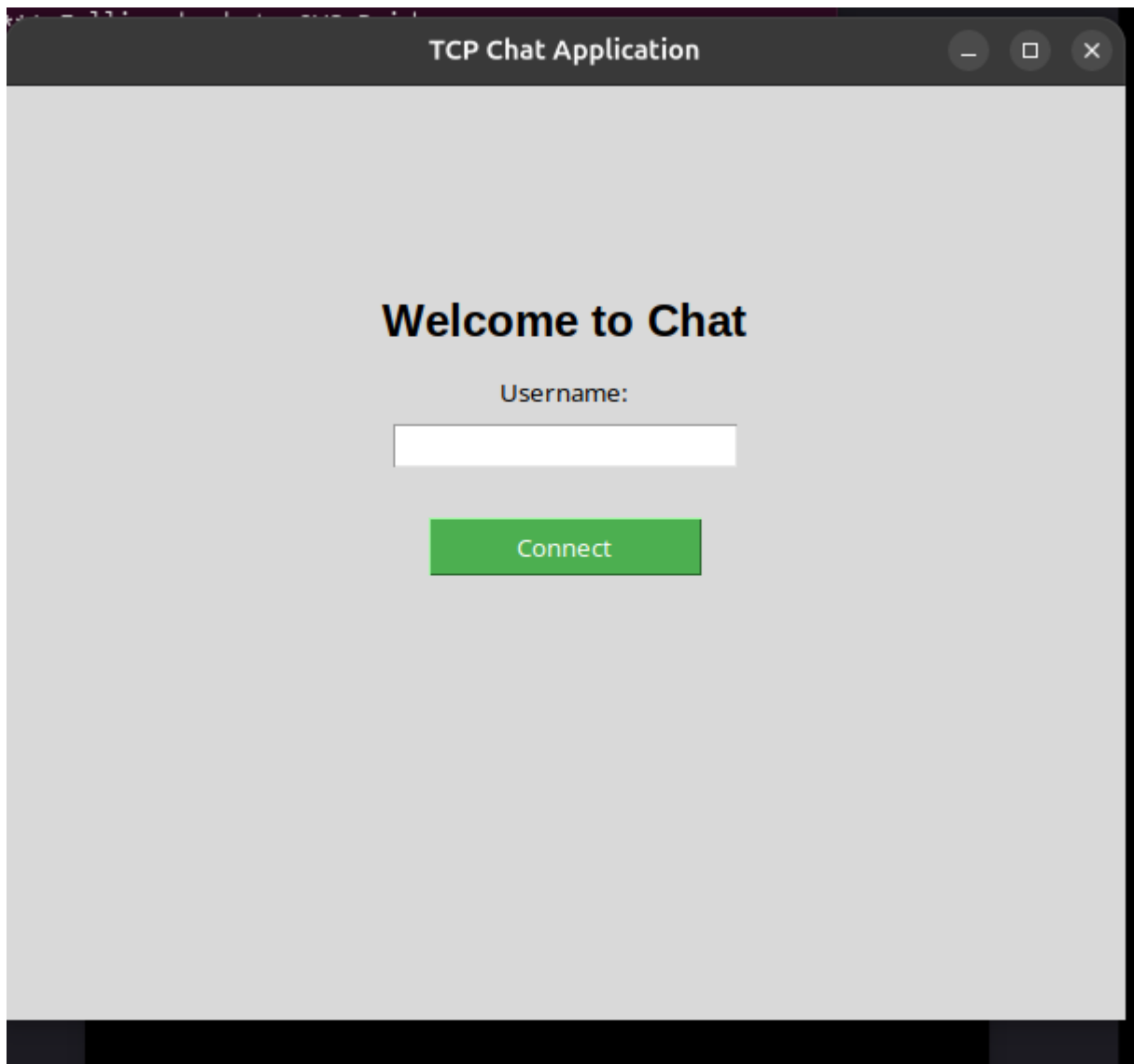
4. Components Reused from Assignment 5

The foundational networking logic from Assignment 5 was preserved and reused. This includes the server's TCP socket initialization, the thread-safe dictionary (protected via RLock) for managing online status and client sockets, the `chat_history.csv` logging functions, and the core routing logic for processing standard broadcasts and `/msg` private commands.

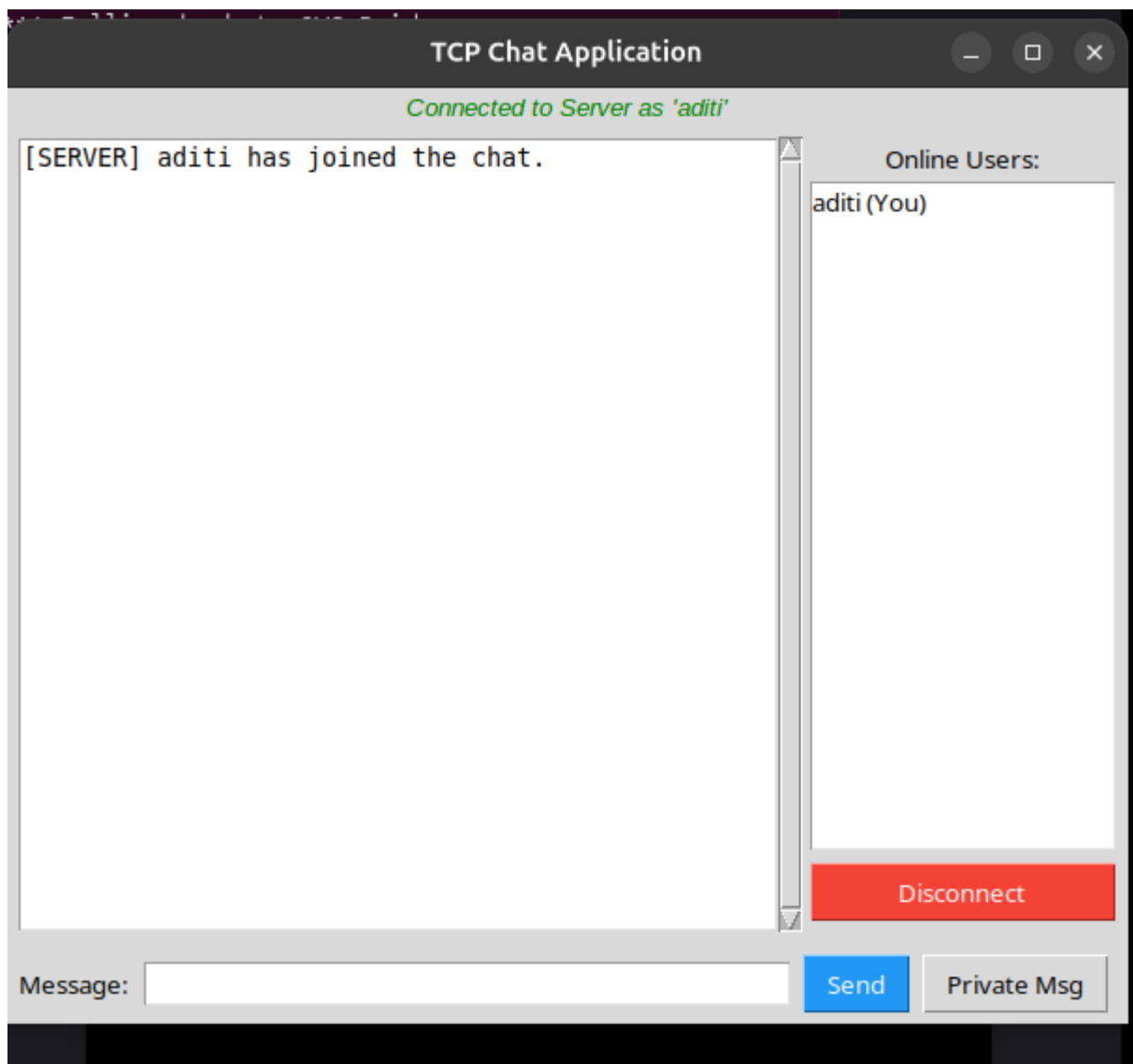
5. GUI Design Decisions

The user interface was designed with a focus on functional simplicity and responsive usability. A dedicated "Private Msg" button was added to automatically format the input field with the `/msg <user>` syntax based on the active selection in the online users list, preventing typos. Furthermore, the server was modified to silently broadcast a `/USERLIST` payload upon any client connection or disconnection, allowing the GUI to dynamically refresh the online users panel without requiring manual user polling.

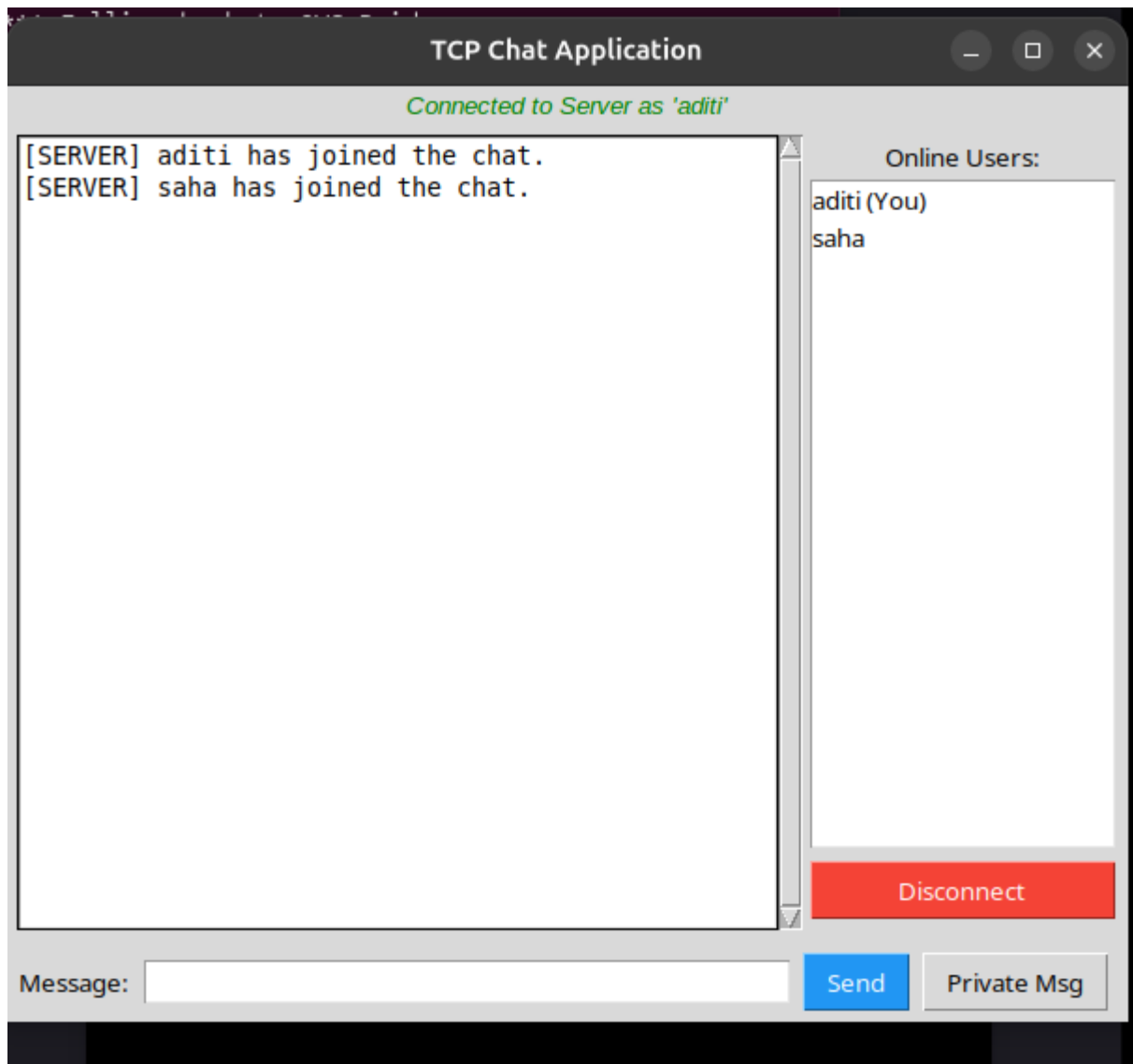
6. Testing Results



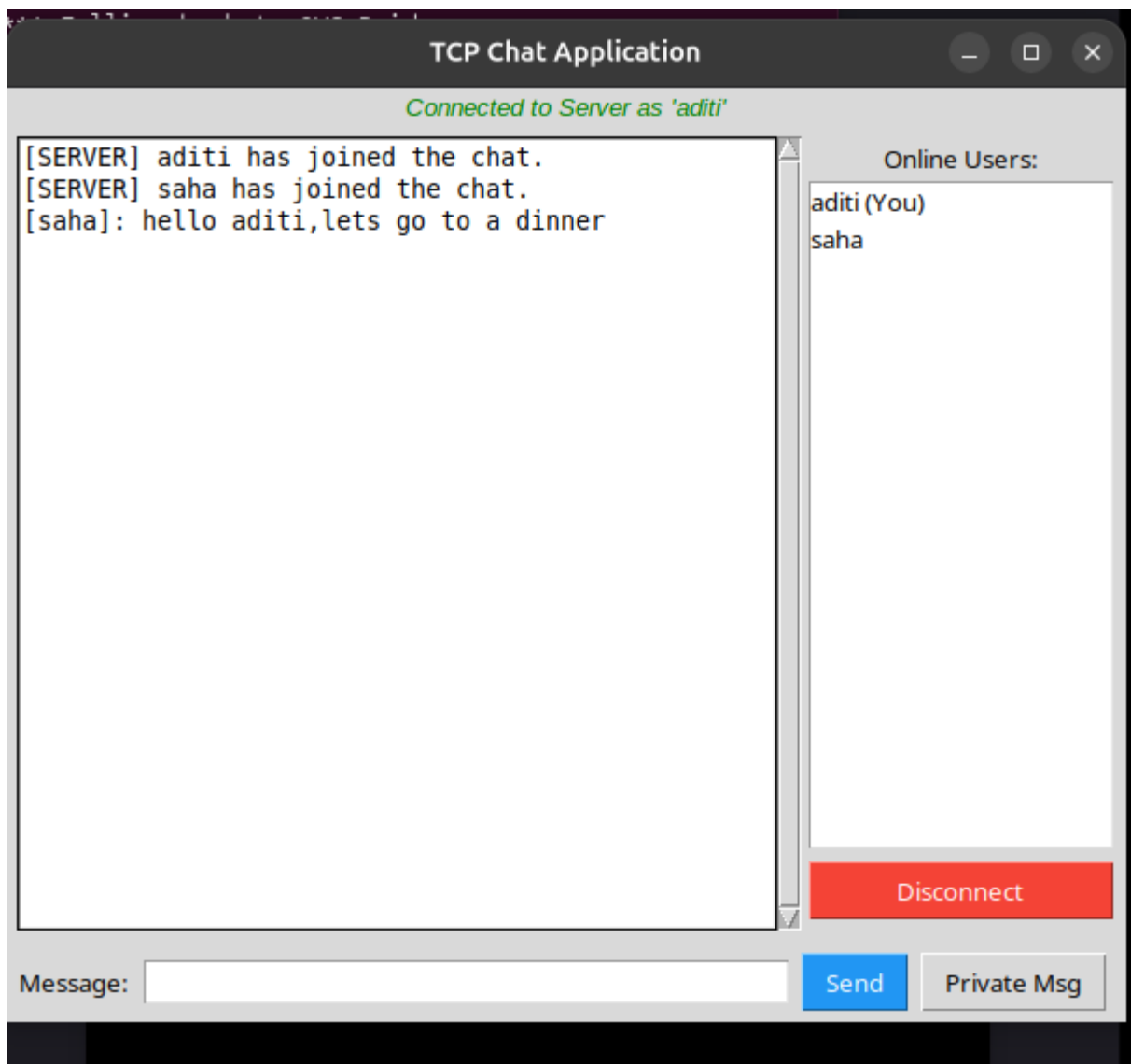
Caption: The initial GUI window prompting the user for a username prior to establishing the TCP connection.



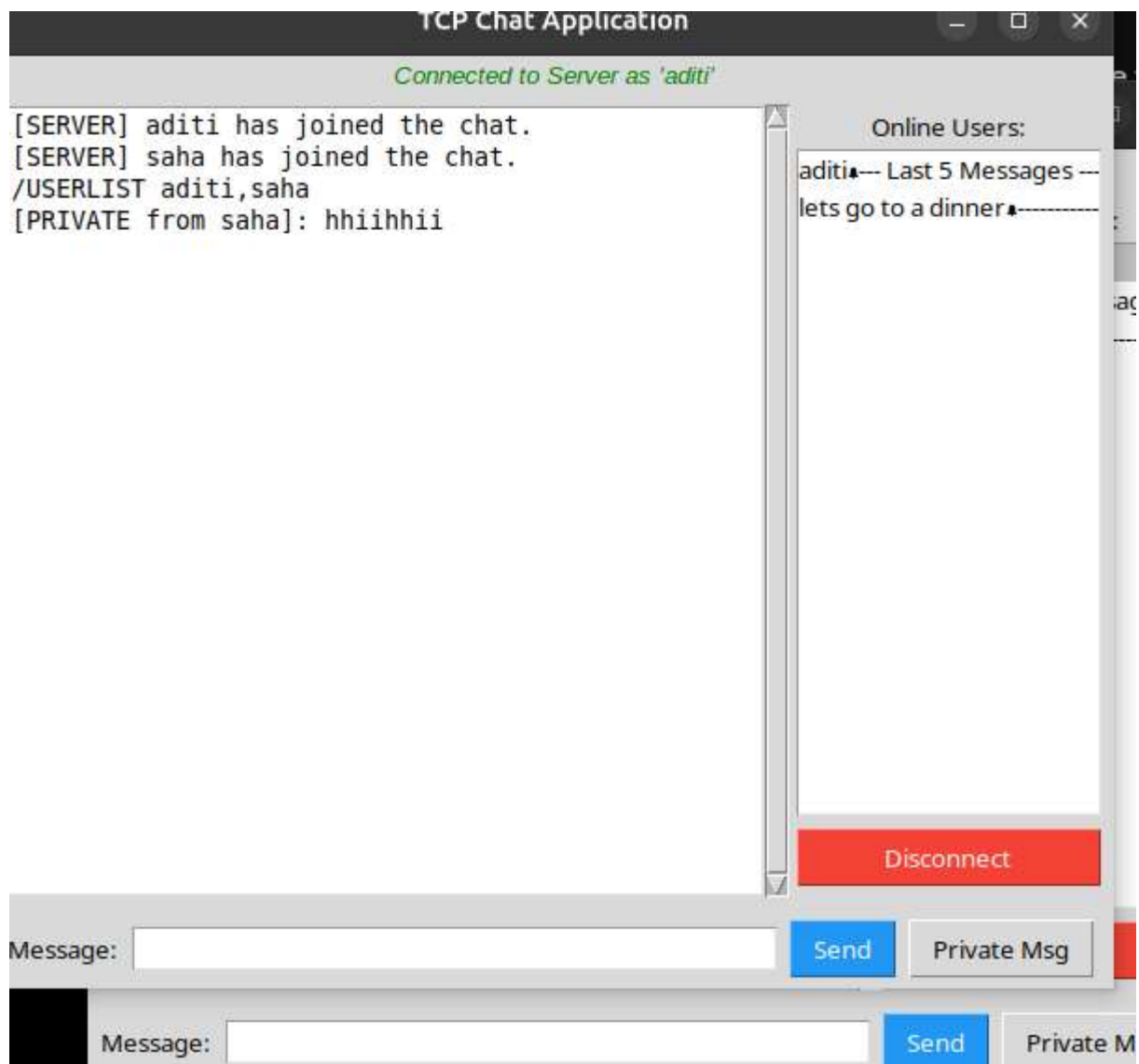
Caption: The main interface showing a successful connection status and the user populated in the online list.



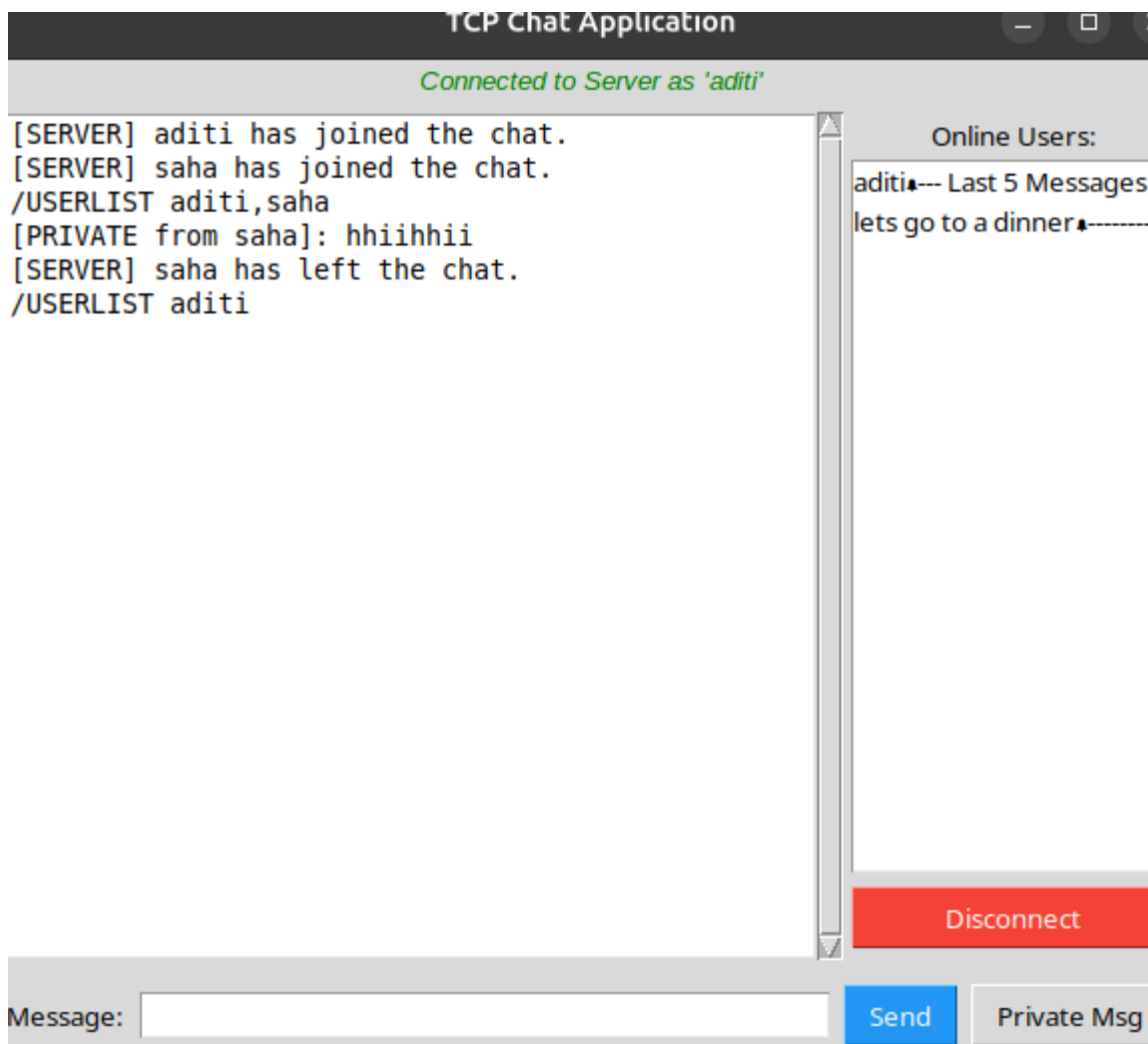
Caption: The server broadcasting a join notification, with the sidebar automatically updating to reflect the new online user.



Caption: A standard message successfully transmitted and rendered in the main chat history.



Caption: A private message routed specifically to the target user, indicated by the [PRIVATE] tag.



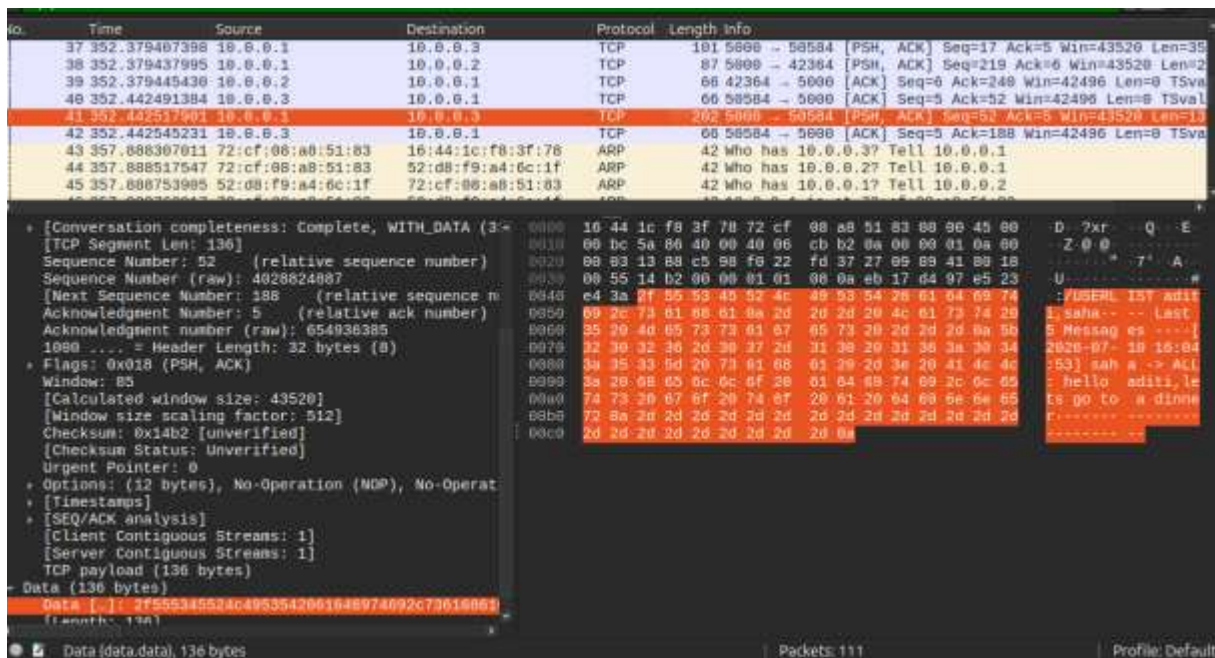
Caption: The interface updating automatically after a user clicks the Disconnect button, removing them from the active user list.

7. Wireshark Verification

No.	Time	Source	Destination	Protocol	Length	Info
17	147.968175618	10.0.0.1	10.0.0.2	TCP	60	5808 → 58038 [PSH, ACK] Seq=1 Ack=17 Win=43520 Len=0
18	147.968218502	10.0.0.2	10.0.0.1	TCP	60	58038 → 5808 [ACK] Seq=1 Ack=17 Win=42496 Len=0
21	158.899782406	10.0.0.2	10.0.0.1	TCP	74	58038 → 5808 [PSH, ACK] Seq=1 Ack=17 Win=42496 Len=0
22	158.899817583	10.0.0.1	10.0.0.2	TCP	60	5808 → 58038 [ACK] Seq=17 Ack=9 Win=43520 Len=0
23	158.900904716	10.0.0.1	10.0.0.2	TCP	184	5808 → 58038 [PSH, ACK] Seq=17 Ack=9 Win=43520 Len=0
24	158.901396614	10.0.0.2	10.0.0.1	TCP	60	58038 → 5808 [ACK] Seq=9 Ack=55 Win=42496 Len=0
28	189.905417051	10.0.0.3	10.0.0.1	TCP	74	58638 → 5808 [SYN] Seq=0 Win=42348 Len=0 MSS=1460
29	189.905441157	10.0.0.1	10.0.0.3	TCP	74	5808 → 58638 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0
30	189.906007078	10.0.0.3	10.0.0.1	TCP	60	58638 → 5808 [ACK] Seq=1 Ack=1 Win=42496 Len=0

Frame 17: Packet, 82 bytes on wire (656 bits), 82 bytes captured
 Ethernet II, Src: 4a:46:fd:8b:01:f2 (4a:46:fd:8b:01:f2), Dst: ae:00:00:00:00:00
 Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
 Transmission Control Protocol, Src Port: 5808, Dst Port: 58038, Seq: 1, Win: 43520, Len: 0
 Data (16 bytes)

Caption: The TCP three-way handshake (SYN, SYN-ACK, ACK) establishing a reliable connection between the GUI client and the server.



Caption: A packet capture demonstrating a standard broadcast message payload being pushed (PSH, ACK) to the network.



Caption: A packet capture showing the precise /msg command payload traversing the network for targeted routing.

8. Reflection Answers

1. Why should networking logic and GUI code be separated?

Network operations, such as `socket.recv()`, are "blocking." If these operations are placed in the main GUI thread, the application's event loop will freeze, causing the window to become entirely unresponsive while waiting for incoming data.

2. Why is a background thread required?

A background thread continuously listens to the network socket for incoming payloads. This allows the main thread to freely render the UI elements, handle button clicks, and process user typing simultaneously without interruption.

3. Which components from Assignment 5 were reused?

The entire server-side routing logic, the thread-safe user management structures, the CSV chat history logging mechanisms, and the core TCP socket initialization phases were successfully reused.

4. How did the GUI improve usability?

It eliminated the need for users to memorize and execute terminal commands. The online user list updates visually and automatically, and private messaging is executed via intuitive point-and-click interactions rather than manual string formatting.

5. Suggest one future enhancement.

Implementing end-to-end encryption (e.g., AES) would significantly enhance security, ensuring that private messages cannot be intercepted and read in plaintext via packet sniffers like Wireshark.

9. Conclusion

The transition from a terminal interface to a graphical desktop application successfully demonstrated the critical importance of multithreading in modern network application design. By isolating blocking network operations into dedicated background threads, the application maintained a highly responsive tkinter interface, resulting in a significantly more intuitive, robust, and user-friendly communication platform.